MULTI-AGENT NLP SYSTEM · CAPSTONE PW26_VRB_07

Hindi Story → Hinglish Screenplay

A Multi-Agent System for Context-Aware Slang Generation

Full Implementation Report

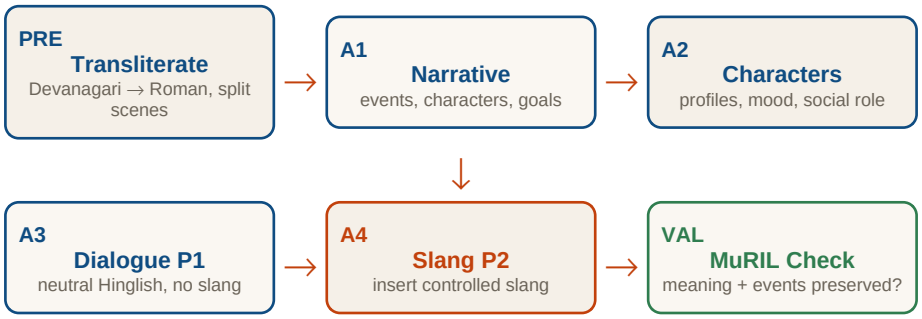
This document explains, in plain language, exactly how the system converts a Devanagari Hindi story into a Hinglish screenplay with natural-sounding slang. It walks through every agent, the transliteration step, how slang is inserted and controlled, how the meaning is checked, and how the training dataset is grown with a validated LLM-augmentation loop. Commands to actually run the program are given separately (outside this document).

Team	Atul · Dia · Anika · Anish
Project code	PW26_VRB_07
Core idea	Devanagari Hindi → romanized Hinglish screenplay + slang, meaning preserved
Backbone	LangGraph multi-agent pipeline · Groq LLM · mT5 slang editor · MuRIL validation

1 The Big Picture

The system is a **pipeline of small specialists**. Instead of asking one large model to "turn this Hindi story into a slangy Hinglish script" in a single step (which loses characters, drifts in meaning, and over-uses the same one or two slang words), we split the job into stages. Each stage does exactly one thing and hands its result to the next. This makes every step inspectable, testable, and separately improvable — which is the whole point of a multi-agent design.

The stages are wired together with **LangGraph** (a state-machine framework). A single shared **scratchpad** object flows through the graph; each agent reads what it needs from the scratchpad and writes back its output, so later agents have full context.



The feedback loop (redo)

After the MuRIL check, the orchestrator makes a decision. If the slang version still means the same thing as the neutral version and still covers the scene's key events, it is **accepted** and assembled into the final script. If it drifted too far, the line is sent **back to Agent 4 to be rewritten** (up to 3 attempts). This accept / redo branch is what makes it a genuine agentic loop rather than a straight line.

Why Groq, and why a local model too

Reasoning-heavy stages (reading the story, writing dialogue) call a hosted LLM on **Groq** (`openai/gpt-oss-20b`, reasoning_effort=low) — fast, no local GPU needed. The slang insertion is the **research target**: a small **mT5 model we fine-tuned ourselves** on our slang dataset. If that local model is absent, Agent 4 automatically falls back to a Groq slang path, so the pipeline always runs.

2 Preprocessing & Transliteration

The input is a Hindi story written in **Devanagari** (■■■■■). Everything downstream works in **romanized** text (Hindi words spelled in the Latin alphabet), because that is how Hinglish is actually written and how our slang dataset is stored. So the first job is a faithful, casual-style transliteration.

Tool used

IndicNLP Library (`indic_nlp_resources`) — a well-known, rule-based Indic-script toolkit. We use its Devanagari→ITRANS transliterator plus its sentence tokenizer. It is rule-based (not a model) so it is deterministic, fast, and free.

The transliteration steps

Each sentence goes through a small, ordered pipeline (`preprocessing/transliterate.py`):

- **Sentence split** — break the story into sentences with IndicNLP's tokenizer.
- **Devanagari → ITRANS** — convert to the ITRANS Roman scheme (a precise but machine-looking romanization, e.g. `kyaa`, `mai.m`).
- **ITRANS → casual** — our rules make it read like real texting: `aa`→`a`, `ii`→`i`, anusvara `.m`→n (so `mai.m`→`main`), lowercase the ITRANS capitals used for retroflex/aspirated sounds.
- **Schwa deletion** — drop the silent trailing 'a' Hindi speakers don't pronounce, plus hand-tuned word fixes (`mujhko`→`mujhe`, `kuchh`→`kuch`).
- **Phonetic smoothing + cleanup** — final specific-word fixes, lowercase, strip noise, keep punctuation.

> **Why rule-based instead of an LLM here**
>
> Transliteration must be **100% faithful** — it must not paraphrase, add, or drop content. A rule engine can't hallucinate; it only re-spells. Using an LLM here would risk silent meaning changes before the story even reaches the agents. The output is also split into **scenes/chunks** so each agent works on one manageable unit at a time.

3 Agent 1 — Narrative Analyst

What it does	Reads the romanized story scene-by-scene and extracts a structured **event chain**: what happens, who is in it, the goal of each scene, the location, and how each pronoun maps to a real character.
Tool / LLM	Groq — `openai/gpt-oss-20b` (reasoning_effort=low).
Input	One transliterated scene + a short summary of the previous scene (for continuity).
Output	A strict JSON object per scene: `key_events, characters, scene_goal, location, narrative_link, pronoun_map`.

Why this agent exists

If you jump straight to writing dialogue, the model forgets who is who and invents events. Agent 1 **pins down the facts first**: a clean list of events and a canonical set of character names. Everything after this is anchored to that structure — which is also what lets us later check whether the final slangy script still covers the same events.

Careful design choices

- **Exactly one JSON object per scene** — the prompt forbids splitting a scene, so scene IDs stay stable and don't multiply.
- **Strict character rule** — only real named individuals count as characters. Groups (*log*/people, *chatron*/students), institutions (*nagar parishad*/city council), and objects (*dayari*/diary) are explicitly excluded, so Agent 2 doesn't build a 'profile' for a diary.
- **pronoun_map** — maps *usne / woh / yeh* to the actual character, fixing the biggest source of confused dialogue.
- **narrative_link** — each scene must reference the previous one, keeping the story coherent across scene boundaries.

4 Agent 2 — Character Analyst

What it does	Builds a **profile for every character**: their social role (peer / elder / authority), an inferred mood/stress, and a slang-level — how casually this person would speak.
Tools / models	Two Hugging Face models, both on CPU: **NER** (`Davlan/bert-base-multilingual-cased-ner-hrl`, ~700 MB) to catch names, and **MuRIL** (`google/muril-base-cased`, ~357 MB) for Indian-language embeddings.
Input	Agent 1's event chain (primary source of names) + the raw romanized scene text.
Output	A `character_profiles` dictionary written to the scratchpad.

How a profile is built

- **Names** come primarily from Agent 1's `characters` list, double-checked with NER on the romanized text (NER catches names Agent 1 missed).
- **Social role** is decided by keyword signals in the text: honorifics like *pitaji, dadaji, uncle, sir, madam, doctor, sahib, ji* mark someone as **elder** or **authority**; everyone else defaults to **peer**.
- A blocklist of **non-character words** (*dayari, kitab, log, shodhakarta, nagar parishad*…) stops objects and groups from being treated as people.

Why the social role matters

The role directly controls slang later. In real Indian speech you do **not** say *"scene set hai yaar"* to your grandfather or a police inspector. So an **elder** or **authority** profile becomes a hard block in Gate 1 — those characters keep clean, respectful Hinglish no matter what. This is how the system stays socially believable.

5 Agent 3 — Dialogue Synthesizer (Pass 1)

What it does	Writes the actual dialogue for each scene — but deliberately **neutral**: correct Hinglish, no slang, no fillers like *yaar* or *bhai* yet.
Tool / LLM	Groq — `openai/gpt-oss-20b`.
Input	Scene goal, location, the narrative link, the speaker's name + social role + current emotion, the listener, and the key events to cover.
Output	Up to 3 dialogue lines, formatted `SPEAKER_NAME: line`.

Why split dialogue into two passes

This is a core design decision. If you ask one model to write slangy dialogue in one shot, it tends to bolt slang onto every line and lose the plot. Instead we do **two passes**:

- **Pass 1 (this agent):** get the *content* right — the right characters saying the right things to cover the scene's events, in clean neutral Hinglish appropriate to their role.
- **Pass 2 (Agent 4):** a separate, controlled step decides *which* lines get slang and *which* slang — without touching meaning.

> **The payoff of separating content from style**
>
> Because Pass 1 is slang-free, we have a clean **neutral reference** for every line. Later, the MuRIL validator compares the slang version against this neutral version to prove the meaning didn't change. Without the two-pass split, there'd be nothing faithful to compare against.

6 The Three-Gate Controller

Before any slang is added, each neutral line passes through **three sequential gates** (`gates/three_gate_controller.py`). A line only becomes eligible for slang if it clears all three. This is the mechanism that keeps slang *tasteful and sparse* instead of spraying it everywhere.

Gate 1 Speaker Eligibility	Blocks **elders** and **authority** figures (and anyone with slang-level 0). Your boss and your grandmother stay formal.
Gate 2 Pragmatic Eligibility	Only **casual / emotional** intents qualify (frustration, excitement, stress, urgency, casual assertion). Formal or purely informational lines are blocked. Intent is labelled by the intent classifier.
Gate 3 Density Control	Blocks a line if the scene already has too much slang (threshold `0.25`). Keeps slang a *seasoning*, not the whole meal.

Intent classifier: a Groq-backed classifier (`agents/intent_classifier.py`) labels each line with one of a fixed set (REQUEST_URGENCY, ASSERT_FRUSTRATION, ASSERT_POSITIVE, EXPRESS_STRESS, CASUAL_ASSERTION, INFORM_NEUTRAL, QUESTION_FORMAL). Results are cached per line to avoid repeat calls.

7 Agent 4 — Slang Rewriter (Pass 2)

What it does	Takes an **eligible** neutral line and inserts one natural slang phrase, keeping the meaning identical.
Primary model	Our own fine-tuned **mT5 slang editor** (`google/mt5-small`, fully fine-tuned on our slang dataset). This is the model that actually *uses* the dataset we built — the research centrepiece.
Fallback	If the local editor is missing or misbehaves, Agent 4 falls back to **Groq**: it **picks the slang in code** (rarity-weighted from the 576-phrase lexicon, matched to emotion + intent) and asks Groq only to *place* it. Choosing in code — not letting the LLM pick — is what breaks the 'everything becomes yaar/bhai' monotony.

The negative result worth publishing

We first tried **IndicBART** for the editor. It has a **Devanagari-only vocabulary**, so on romanized Hinglish it collapsed — copying the input, emitting sentinel tokens (`<extra_id_0>`), then Gujarati garbage. Switching to a **full fine-tune of mT5-small** with an edit-weighted loss fixed it. That 'collapse cascade' is a clean, documentable finding for the paper.

8 Validation & the Redo Loop

After slang is inserted, the **MuRIL Validator** (`evaluation/muril_validator.py`) checks two things before the line is allowed into the final script:

- **Meaning preserved?** — It embeds both the neutral (Pass 1) line and the slang (Pass 2) line with **MuRIL** and measures cosine similarity. If similarity ≥ `0.82`, the meaning is considered intact.
- **Events still covered?** — It checks that the scene's key events (from Agent 1) are still recoverable from the dialogue (event-recall ≥ `0.5`).

The orchestrator turns this into an **accept / redo** decision. Pass → the line is assembled into the screenplay. Fail → the line goes **back to Agent 4** to try a different, lighter slang — up to `MAX_REWRITE_ATTEMPTS = 3`. This closes the agentic loop: the system critiques and repairs its own output.

> **Why MuRIL specifically**
>
> MuRIL (Multilingual Representations for Indian Languages) is trained on Indian languages **including transliterated text**, so it understands that a neutral Hindi line and its slangy cousin mean the same thing — something generic English embedders miss. It's also small enough to run on a laptop CPU.

Evaluation metrics (offline scoring)

Separately from the live loop, `evaluation/` scores saved runs with **reference-free metrics**: slang coverage / density / validity / diversity, **event recall**, the **Code-Mixing Index (CMI)** after Gambäck & Das, and an optional **LLM-judge** that grades emotional fit and slang placement out of 5. These are what go into the results tables in the paper.

9 Dataset Augmentation (the validated LLM loop)

The slang editor (Agent 4) is only as good as its training data. Our hand-cleaned base dataset covers a limited slice of slang and contexts, and naive LLM generation drifts badly (it over-uses *bindaas*, breaks grammar, and mislabels). So we grow the dataset with a **tightly-controlled, validated augmentation pipeline** (`data_prep/augment_slang_dataset.py`). Every new row is generated by an LLM and then **put on trial** before it is allowed in.

Step by step: what happens per row

1	**Pick a target slang**	Not the LLM's choice — we draw the target slang from a 576-phrase lexicon, rarity-w
2	**Choose a mode**	VARIANT (~85%): take a real, guaranteed-grammatical neutral line from the clean data
3	**Call gemma (the generator)**	Send an INSERT or NOVEL prompt asking for JSON only: the slang inserted naturally, Roman
4	**Structural checks**	Reject automatically if: it contains Devanagari, the slang isn't actually present, nothing changed, wrong
5	**LLM-JUDGE (the trial)**	A second gemma call judges the survivor on five booleans, all of which must be
6	**Keep or quarantine**	Only rows that pass every check are written to the dataset. Everything else is logged to a transparent <

The two generation prompts (what gemma is actually told)

VARIANT / INSERT prompt (insert a chosen slang into a real neutral line):

```
Insert the slang expression "{slang}" into this neutral Hinglish line so it
sounds natural and casual, WITHOUT changing the meaning.
Return ONLY JSON: {"output": "<the line with '{slang}' inserted naturally>"}
Rules: romanized Roman script only; keep all facts; one line;
       'output' must contain "{slang}".
Neutral line: {neutral}
```

JUDGE prompt (the five-way boolean trial every survivor faces):

```
You are a strict data-quality judge for a Hindi-Hinglish slang dataset.
Neutral: {neutral}
Slang output: {output}
Answer ONLY JSON with booleans:
{"is_hinglish": ...,  "grammatical": ...,  "meaning_preserved": ...,
 "intent_match": ...,  "is_real_slang": ...}
```

Augmentation: tokens, calls, and acceptance

Each accepted row costs **two** LLM calls (one to generate, one to judge); a rejected row that reaches the judge also costs two, and one rejected early on structure costs one. Prompts are short — roughly **120–180 input tokens** and **~40 output tokens** (JSON only) per call — which is deliberately why the whole run stays cheap.

Metric	Value	Meaning
Rows kept	**1,500**	clean, validated slang pairs added to the dataset
Attempts	**4,177**	candidate rows generated to reach 1,500 keepers
Acceptance rate	**~36%**	roughly 1 in 3 candidates survives every check
Distinct slang used	**486**	spread across the vocabulary (anti-concentration working)
Rejected: LLM-judge	**2,041**	biggest filter — failed one of the 5 quality booleans
Rejected: duplicate	**331**	output already existed in the dataset
Rejected: semantic drift	**302**	strayed too far from the neutral meaning
Rejected: other	**36**	slang absent (15), wrong length (19), no change (2)
Generator model	**gemma-4-31b**	on Cerebras (backend-swappable to Groq)
Rate limits respected	**5/min**	paced with a 13s min-interval; auto-resume from checkpoints

> **Why an ~36% acceptance rate is a feature, not a bug**
>
> A high acceptance rate would mean the judge is lazy. Throwing away nearly two-thirds of candidates — with the LLM-judge alone rejecting 2,041 — is exactly the strict filtering that keeps the 'platinum' dataset clean. The cost is bounded because prompts are tiny and JSON-only, and the run checkpoints every 5 rows so it can resume after any interruption (we hit disk-full and rate limits mid-run and simply continued).

Resilience built in: the writer saves atomically every 5 rows (temp file + replace, so the CSV is never half-written), and on restart it **reloads existing rows and continues** rather than redoing work. Rate-limit errors (HTTP 429) are caught and paced automatically.

10 Running It & Showing the Demo

The exact shell commands are provided **separately, alongside this document** (so they can be copied and run directly). This section explains **what** to run and **how to present it** in VS Code for the demo.

What running the pipeline looks like

- You point the runner at a Hindi story file and give the run a session name.
- The **first run** downloads ~1 GB of models (the name-detector + MuRIL) once and caches them; later runs are fast.
- You do **not** need the trained slang editor present — if it's absent the pipeline uses the Groq slang path automatically, so the demo always works.
- Two files land in `outputs/`: the final screenplay (`<session>_script.txt`) and a full step-by-step trace (`<session>_trace.json`).

How to show the demo in VS Code (suggested flow)

1	Open the project folder in VS Code and open an integrated terminal (Terminal → New Terminal). Activate the conda env fi
2	Open the input story side-by-side: data/raw/test_story_1.txt. Show the audience it's plain <b>D
3	Run the pipeline command (given below the doc). Let the terminal stream the stages — the audience sees each age
4	Open outputs/<session>_script.txt in an editor tab: the final <b>Hinglish screenplay with s
5	Open outputs/<session>_trace.json to prove it's real: show the neutral Pass-1 line, the sla
6	(Optional) Run the evaluator to show the metrics table (slang density, CMI, event recall), and open the augmented dataset

> **Demo tip**
>
> The single most convincing moment is the **side-by-side**: Devanagari input on the left, slangy Hinglish screenplay on the right, and the trace JSON showing the meaning-check score in the middle. That visually proves the three things that matter — it transliterated, it added believable slang, and it *proved* the meaning survived.

The runnable commands (setup, run, evaluate, augment) are listed outside this PDF so you can copy them straight into the VS Code terminal.